

A Proposal to Add an Intrusive Smart Pointer to the C++ Standard Library

Audience: SG14, LEWG

Reply-To: Isabella Muerte <isabella.muerte@mnmlstc.com (mailto:isabella.muerte@mnmlstc.com)> Bryce Adelstein Lelbach <balelbach@lbl.gov (mailto:balelbach@lbl.gov)>

Author: Isabella Muerte <isabella.muerte@mnmlstc.com (mailto:isabella.muerte@mnmlstc.com)>

Date: 2016-10-15

ID: P0468R0

Table of Contents

- 1 Abstract
- 2 Motivation and Scope
- 3 Impact On the Standard
- 4 Frequently Asked Questions
 - 4.1 How does boost::intrusive_ptr not meet the needs of modern C++?
 - 4.2 Is retain_ptr atomic?
 - 4.3 Why is it called retain_ptr and not intrusive_ptr?
 - 4.4 Does retain_ptr support allocators?
 - 4.5 Can retain_ptr be constexpr?
 - 4.6 Why does retain_ptr use detach instead of release like unique_ptr?
 - 4.7 Why provide retain_t?
 - 4.8 Can retain_traits store state?
 - 4.9 Why not just wrap a unique_ptr with a custom deleter?
- 5 Technical Specification
 - 5.1 atomic_reference_count<T> and reference_count<T>
 - 5.2 retain_t
 - 5.3 retain_traits<T>
 - 5.4 retain_ptr<T, R>
 - 5.4.1 retain_ptr constructors
 - 5.4.2 retain_ptr destructor
 - 5.4.3 retain_ptr assignment
 - 5.4.4 retain_ptr observers
 - 5.4.5 retain_ptr modifiers
 - 5.4.6 retain_ptr specialized algorithms
- 6 Examples
 - 6.1 Futures
- 7 Acknowledgements
- 8 References

1 Abstract

I propose a new smart pointer (**`retain_ptr<T, R>`**) whose reference count is stored inside of a managed object (i.e., intrusively). I believe that this will reduce the complexity of implementing objects to easily manage C and C++ APIs whose object lifetimes rely on a reference count stored directly within a given object.

2 Motivation and Scope

There are a wide variety of C and C++ APIs that rely on reference counting, but either because of the language (C) or the age of the library (C++), they are unable to be safely used with either `std::unique_ptr<T>` or `std::shared_ptr<T>`. In addition, existing intrusive smart pointers such as `boost::intrusive_ptr<T>` [1], Microsoft's `ComPtr<T>` [2], or WebKit's `WTF::RefPtr<T>` [3] do not meet the needs of modern C++ smart pointers or APIs, and this paper attempts to solve these shortcomings in an extensible and future proof manner.

The users that would get the best use out of this type are those that work on systems that rely on reference counting (usually interacting with C APIs).

Additionally, with an intrusive smart pointer, one can implement a non-atomic `shared_ptr` and `weak_ptr`. Furthermore, implementing ones own promise and future is possible, which will be useful with the coming Coroutines TS.

A reference implementation of `retain_ptr<T>`, along with an example of its use, can be found on github (<https://github.com/slurps-mad-rips/retain-ptr>).

3 Impact On the Standard

retain_ptr<T, R> would ideally be available in the **<memory>** standard header. It is a pure extension to the C++ standard library and can be implemented using any conforming C++14 or C++11 compiler with very little effort. See the Technical Specification for interface and behavior details.

4 Frequently Asked Questions

Several common questions regarding the design of `retain_ptr<T, R>` can be found below.

4.1 How does `boost::intrusive_ptr` not meet the needs of modern C++?

`boost::intrusive_ptr<T>` has had nearly the same interface since its introduction in 2001 by Peter Dimov. Furthermore, `boost::intrusive_ptr` has several failings in its API that cannot be changed from without breaking compatability. When constructing a `boost::intrusive_ptr<T>`, by default it increments the reference count. This is because of its `intrusive_ref_count` mixin, which starts with a reference count of 0 when it is default constructed. Out of all the libraries I tried to look at, this was the one instance where an object required it be incremented after construction. This should be the exception, and this proposal rectifies this with its equivalent mixins, which start with a reference count of 1.

Additionally, `boost::intrusive_ptr` does not have the ability to "overload" its `pointer` type member, requiring some additional work when interfacing with C APIs (e.g., `boost::intrusive_ptr<decltype(*declval<cl_mem>())>`).

Furthermore, `boost::intrusive_ptr` relies on ADL calls of two functions: `intrusive_add_ref` and `intrusive_release`. While this approach is fine in most cases, it does remove the ability to easily "swap out" the approach used when incrementing or decrementing the reference count (e.g., logging when reference count reaches 0, but not when in a production environment). This approach also uses terms found in Microsoft's COM. While this isn't an issue per se, it would be odd to have functions with those names found in the standard.

4.2 Is `retain_ptr` atomic?

`retain_ptr<T>` is only atomic in its reference count increment and decrement if the object it manages is itself atomic in its reference count operations.

4.3 Why is it called `retain_ptr` and not `intrusive_ptr`?

`retain_ptr<T, R>` diverges from the design of `boost::intrusive_ptr<T>`. It was decided to change the name so as to not cause assumptions of `retain_ptr<T, R>` interface and behavior.

Some additional names that might be considered (for bikeshedding) are:

- `extend_ptr`
- `counted_ptr`
- `borrow_ptr`
- `mutual_ptr`
- `joint_ptr`

Comedy Option:

- `auto_ptr`

4.4 Does `retain_ptr` support allocators?

`retain_ptr` itself does not support allocators, however the object whose lifetime it extends can.

4.5 Can `retain_ptr` be constexpr?

Possibly, however the author questions the usefulness for a constexpr capable intrusive smart pointer, as most use cases are intended for migrating existing non-constexpr interfaces, and for types that simply cannot be constexpr, such as incomplete types and polymorphic classes.

4.6 Why does `retain_ptr` use `detach` instead of `release` like `unique_ptr`?

`retain_ptr<T>` diverges from common smart pointer functions that release ownership of their managed object via a function `release()`, which returns the object in its current state, and places the `retain_ptr` into an empty state. `retain_ptr<T>` opts to use the name `detach()` for semantic reasons. Many objects that might be managed by `retain_ptr<T>` tend to use a function or operation named `release` to decrement the internal reference count. To reduce confusion for implementers and those curious enough to look under the hood, a different name was chosen (i.e., `detach()`). This name was also used by `boost::intrusive_ptr`.

A `retain_ptr<T>` does not *own* the object it manages. Rather it is *extending* ownership. When we use `detach`, we aren't telling the `retain_ptr<T>` to release ownership to the caller. Instead we are expressing our desire for the `retain_ptr<T>` to *detach* its management of the object it stores. Other possible names for `detach()` that could be considered are `disengage`, `discard`, and `withdraw`.

4.7 Why provide `retain_t`?

`retain_t` (and its instance `retain`) are currently used to represent the case where a `retain_ptr` needs to *extend* (i.e., retain) a pointer to an object. This mostly comes into play when interacting with APIs that return a borrowed reference to an object without incrementing its reference count to begin with. Additionally, an `enum class retain : bool { no, yes }` would technically be possible, but this would be the first time

such an API is placed into the standard library.

The name of this type is available for bikeshedding however. Some other less elegant names include:

- `retain_element_t`
- `extend_element_t`
- `retainobj_t`
- `extendobj_t`
- `extend_t`

4.8 Can `retain_traits` store state?

No. Any important state regarding the object or how it is retained, can be stored in the object itself. For example, if the reference count needs to be external from the object, `std::shared_ptr` would be a better choice.

4.9 Why not just wrap a `unique_ptr` with a custom deleter?

This is an extraordinary amount of code that would not be guaranteed to have a homogenous interface across different libraries and implementations. For example, using `retain_ptr` with an OpenCL context object (without checking for errors in both implementations) is as simple as:

```
1 struct context_traits {
2     using pointer = cl_context;
3     static void increment (pointer p) { clRetainContext(p); }
4     static void decrement (pointer p) { clReleaseContext(p); }
5 };
6
7 struct context {
8     using handle_type = retain_ptr<cl_context, context_traits>;
9     using pointer = handle_type::pointer;
10    context (pointer p, retain_t) : handle(p, retain) { }
11    context (pointer p) : handle(p) { }
12 private:
13    handle_type handle;
14 };
```

Using the `unique_ptr` approach requires more effort. In this case, it is twice as long to get the same functionality:

```

1 struct context_deleter {
2     using pointer = cl_context;
3     void increment (pointer p) const {
4         if (p) { clRetainContext(p); } // retain_ptr checks for null for us
5     }
6     void operator () (pointer p) const { clReleaseContext(p); }
7 };
8
9 struct retain_t { };
10 constexpr retain_t retain { };
11
12 struct context {
13     using handle_type = unique_ptr<cl_context, context_deleter>;
14     using pointer = handle_type::pointer;
15
16     context (pointer p, retain_t) :
17         context(p)
18     { handle.get_deleter().increment(handle.get()); }
19
20     context (pointer p) : handle(p) { }
21
22     context (context const& that) :
23         handle(that.handle.get())
24     { handle.get_deleter().increment(handle.get()) }
25
26     context& operator = (context const& that) {
27         context(that.handle.get(), retain).swap(*this);
28         return *this;
29     }
30
31 private:
32     handle_type handle;
33 };

```

As we can see, using **retain_ptr<T>** saves effort, allowing us in most cases to simply rely on the "rule of zero" for constructor management. It will also not confuse/terrify potential maintainers of code bases where objects construct a **unique_ptr** with the raw pointer of another (and ownership is not transferred).

5 Technical Specification

A *retain pointer* is an object that extends the lifetime of another object (which in turn manages its own disposition) and manages that other object through a pointer. Specifically, a retain pointer is an object r that stores a pointer to a second object p and will cease to extend the lifetime of p when r is itself destroyed (e.g., when leaving a block scope). In this context, r is said to *retain* p , and p is said to be a *self disposing object*.

When p 's lifetime has reached its end, p will dispose of itself as it sees fit. The conditions regarding p 's lifetime is handled by some count c that p comprehends, but is otherwise not directly accessible to r .

The mechanism by which r retains and manages the lifetime of p is known as p 's associated *retainer*, a stateless object that provides mechanisms for r to increment, decrement, and (optionally) provide access to c . In this context, r is able to *increment* c , *decrement* c , or access the c of p .

Let the notation $r.p$ denote the pointer stored by r . Upon request, r can Furthermore, r can explicitly choose to increment c when $r.p$ is replaced.

Additionally, r can, upon request, *transfer ownership* to another retain pointer $r2$. Upon completion of such a transfer, the following postconditions hold:

- $r2.p$ is equal to the pre-transfer $r.p$, and
- $r.p$ is equal to **nullptr**

Furthermore, r can, upon request, *extend ownership* to another retain pointer $r2$. Upon completion of such an extension, the following postconditions hold:

- $r2.p$ is equal to $r.p$
- c has been incremented by 1

Each object of a type **U** instantiated from the **retain_ptr** template specified in this proposal has the lifetime extension semantics specified above of a retain pointer. In partial satisfaction of these semantics, each such **U** is **MoveConstructible**, **MoveAssignable**, **CopyConstructible** and **CopyAssignable**. The template parameter **T** of **retain_ptr** may be an incomplete type. (Note: The uses of **retain_ptr** include providing exception safety for self disposing objects, extending management of self disposing objects to a function, and returning self disposing objects from a function.)


```

class atomic_reference_count<T>;
class reference_count<T>;

class retain_t;

template <class T> struct retain_traits;

template <class T, class R = retain_traits<T>> class retain_ptr;

template <class T, class R>
void swap (retain_ptr<T, R>& x, retain_ptr<T, R>& y) noexcept;

template <class T, class R>
bool operator == (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator != (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator >= (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator <= (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator > (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator < (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

template <class T, class R>
bool operator == (retain_ptr<T, R> const& x, nullptr_t) noexcept;
template <class T, class R>
bool operator != (retain_ptr<T, R> const& x, nullptr_t) noexcept;
template <class T, class R>
bool operator >= (retain_ptr<T, R> const& x, nullptr_t) noexcept;
template <class T, class R>
bool operator <= (retain_ptr<T, R> const& x, nullptr_t) noexcept;
template <class T, class R>
bool operator > (retain_ptr<T, R> const& x, nullptr_t) noexcept;
template <class T, class R>
bool operator < (retain_ptr<T, R> const& x, nullptr_t) noexcept;

template <class T, class R>
bool operator == (nullptr_t, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator != (nullptr_t, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator >= (nullptr_t, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator <= (nullptr_t, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator > (nullptr_t, retain_ptr<T, R> const& y) noexcept;
template <class T, class R>
bool operator < (nullptr_t, retain_ptr<T, R> const& y) noexcept;

```

5.1 atomic_reference_count<T> and reference_count<T>

atomic_reference_count<T> and **reference_count<T>** are mixin types, provided for user defined types that simply rely on **new** and **delete** to have their lifetime extended by **retain_ptr**. The template parameter **T** is intended to be the type deriving from **atomic_reference_count** or **reference_count** (a.k.a. the curiously repeating template pattern, CRTP).

```

template <class T>
struct atomic_reference_count {
    friend retain_traits<T>;
protected:
    atomic_reference_count () = default;
private:
    atomic<uint_least64_t> count { 1 }; // provided for exposition
};

template <class T>
struct reference_count {
    friend retain_traits<T>;
protected:
    reference_count () = default;
private:
    uint_least64_t count { 1 }; // provided for exposition
};

```

5.2 retain_t

retain_t is a sentinel type, with a constexpr instance **retain**.

```

namespace std {
    struct retain_element_t { constexpr retain_element_t () = default; }
    constexpr retain_t retain { };
}

```

5.3 retain_traits<T>

The class template **retain_traits** serves as the default traits object for the class template **retain_ptr**. Unless **retain_traits** is specialized for a specific type, the template parameter **T** must inherit from either **atomic_reference_count<T>** or **reference_count<T>**. In the event that **retain_traits** is specialized for a type, the template parameter **T** may be an incomplete type.

```

namespace std {
    template <class T> struct retain_traits {
        static void increment (atomic_reference_count<T>*) noexcept;
        static void decrement (atomic_reference_count<T>*) noexcept;
        static long use_count (atomic_reference_count<T>*) noexcept;

        static void increment (reference_count<T>*) noexcept;
        static void decrement (reference_count<T>*) noexcept;
        static long use_count (reference_count<T>*) noexcept;
    };
}

```

static void increment (atomic_reference_count<T>* ptr) noexcept;

effects: Increments the internal reference count for *ptr* with **memory_order_acq_rel**
postcondition: **ptr->count** has been incremented by 1.

static void decrement (atomic_reference_count<T>* ptr) noexcept;

effects: Decrements the internal reference count for *ptr* with **memory_order_acq_rel**. If the internal reference count of *ptr* reaches 0, it is disposed of via **delete**.

static long use_count (atomic_reference_count<T>* ptr) noexcept;

returns: The internal reference count for *ptr* with **memory_order_acquire**.

static void increment (reference_count<T>* ptr) noexcept;

effects: Increments the internal reference count for *ptr* by 1.

static void decrement (reference_count<T>* ptr) noexcept;

effects: Decrements the internal reference for *ptr* by 1. If the count reaches 0, *ptr* is disposed of via **delete**.

```
static long use_count (reference_count<T>* ptr) noexcept;
```

returns: The reference count for *ptr*.

5.4 retain_ptr<T, R>

The default type for the template parameter **R** is **retain_traits**. A client supplied template argument **R** shall be an object with non-member functions for which, given a **ptr** of type **unique_ptr<T, R>::pointer**, the expressions **R::increment(ptr)** and **R::decrement(ptr)** are valid and has the effect of retaining or disposing of the pointer as appropriate for that retainer.

If the *qualified-id* **R::pointer** is valid and denotes a type, then **retain_ptr<T, R>::pointer** shall be synonymous with **R::pointer**. Otherwise **retain_ptr<T, R>::pointer** shall be a synonym for **element_type***. The type **retain_ptr<T, R>::pointer** shall satisfy the requirements of **NullablePointer**.

```
template <class T, class R=retain_traits<T>>
struct retain_ptr {
    using element_type = T;
    using traits_type = R;
    using pointer = /* see below */

    retain_ptr (pointer, retain_t) noexcept(/* see below */);
    explicit retain_ptr (pointer) noexcept;
    retain_ptr (nullptr_t) noexcept : retain_ptr() { }

    retain_ptr (retain_ptr const&) noexcept(/* see below */);
    retain_ptr (retain_ptr&&) noexcept;
    retain_ptr () noexcept;
    ~retain_ptr () noexcept(/* see below */);

    retain_ptr& operator = (retain_ptr const&) noexcept(/* see below */);
    retain_ptr& operator = (retain_ptr&&) noexcept;

    retain_ptr& operator = (nullptr_t) noexcept;

    void swap (retain_ptr&) noexcept;

    explicit operator pointer () const noexcept;
    explicit operator bool () const noexcept;

    element_type& operator * () const noexcept;
    pointer operator -> () const noexcept;
    pointer get () const noexcept;

    long use_count () const noexcept(/* see below */);
    bool unique () const noexcept(/* see below */);

    [[nodiscard]] pointer detach () noexcept;

    void reset (pointer, retain_t) noexcept(/* see below */);
    void reset (pointer p = pointer { }) noexcept(/* see below */);
};
```

5.4.1 retain_ptr constructors

```
retain_ptr (pointer p, retain_t) noexcept(maybe);
```

effects: Constructs a **retain_ptr** that retains **p**, initializing the stored pointer with **p**, and increments the reference count of **p** if **p != nullptr**.

postconditions: **get() == p**.

remarks: This constructor is only specified **noexcept** if **traits_type::increment** function is also specified **noexcept**. If an exception is thrown during this operation, this constructor will have no effect.

```
explicit retain_ptr (pointer p) noexcept;
```

effects: Constructs a **retain_ptr** that retains **p**, initializing the stored pointer with **p**.

postconditions: **get() == p**

remarks: **p**'s reference count remains untouched.

retain_ptr () noexcept;

effects: Constructs a **retain_ptr** object that retains nothing, value-initializing the stored pointer.
postconditions: **get() == nullptr**

retain_ptr(retain_ptr const& r) noexcept(maybe);

effects: Constructs a **retain_ptr** by extending management from **r** to ***this**.
postconditions: **get() == r.get()**
remarks: This constructor is only specified **noexcept** if **traits_type::increment** function is specified **noexcept**. If an exception is thrown during this operation, this constructor will have no effect.

retain_ptr(retain_ptr&& r) noexcept;

effects: Constructs a **retain_ptr** by transferring management from **r** to ***this**.
postconditions: **get()** yields the value **r.get()** yielded before the construction.

5.4.2 retain_ptr destructor

~retain_ptr() noexcept(maybe);

effects: If **get() == nullptr**, there are no effects. Otherwise, **traits_type::decrement(get())**.
remarks: This destructor is only specified **noexcept** if the **traits_type::decrement** function is specified **noexcept**

5.4.3 retain_ptr assignment

retain_ptr& operator = (retain_ptr const& r) noexcept(maybe);

effects: Extends ownership from **r** to ***this** as if by calling **reset(r.get(), retain)**.
returns: ***this**
remarks: This operator is only specified **noexcept** if both **traits_type::increment** and **traits_type::decrement** functions are specified **noexcept**.

retain_ptr& operator = (retain_ptr&& r) noexcept;

effects: Transfers ownership from **r** to ***this** as if by calling **reset(r.detach())**
returns: ***this**

retain_ptr& operator = (nullptr_t) noexcept;

effects: **reset()**
postconditions: **get() == nullptr**
returns: ***this**

5.4.4 retain_ptr observers

element_type& operator * () const noexcept;

requires: **get() != nullptr**
returns: ***get()**

pointer operator -> () const noexcept;

requires: **get() != nullptr**
returns: **get()**
note: use typically requires that **element_type** be a complete type.

pointer get () const noexcept;

returns: The stored pointer

explicit operator pointer () const noexcept;

returns: **get()**

```
explicit operator bool () const noexcept;
```

returns: `get() != nullptr`

```
long use_count () const noexcept(maybe);
```

returns: Value representing the current reference count of the stored pointer. If `traits_type::use_count(get())` is not a valid expression, `-1` is returned. If `get() == nullptr` `0` is returned

remarks: This observer is only specified **noexcept** if `traits_type::use_count` is specified **noexcept**. Unless otherwise specified, the value returned should be considered stale.

```
bool unique () const noexcept(maybe);
```

returns: `use_count() == 1`

remarks: This observer is only specified **noexcept** if `traits_type::use_count` is specified **noexcept**. Unless otherwise specified, the value returned should be considered stale.

5.4.5 retain_ptr modifiers

```
[[nodiscard]] pointer detach () noexcept;
```

postcondition: `get() == nullptr`

returns: The value `get()` had at the start of the call to **detach**

```
void reset (pointer p, retain_t) noexcept(maybe);
```

effects: Assigns **p** to the stored pointer, and then if the old value of the stored pointer **old_p**, was not equal to **nullptr**, calls `traits_type::decrement`. Then if **p** is not equal to **nullptr**, `traits_type::increment` is called.

postconditions: `get() == p`

remarks: This modifier is only specified **noexcept** if both `traits_type::decrement` and `traits_type::increment` are specified **noexcept**.

```
void reset (pointer p = pointer { }) noexcept(maybe);
```

effects: Assigns **p** to the stored pointer, and then if the old value of the stored pointer, **old_p**, was not equal to **nullptr**, calls `traits_type::decrement`.

postconditions: `get() == p`

remarks: This modifier is only specified **noexcept** if `traits_type::decrement` is specified **noexcept**.

```
void swap (retain_ptr& r) noexcept;
```

effects: Invokes **swap** on the stored pointers of ***this** and **r**.

5.4.6 retain_ptr specialized algorithms

```

template <class T, class R>
void swap (retain_ptr<T, R>&, retain_ptr<T, R>&) noexcept;

template <class T, class R>
bool operator == (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator != (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator >= (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator <= (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator > (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator < (retain_ptr<T, R> const&, retain_ptr<T, R> const&) noexcept;

template <class T, class R>
bool operator == (retain_ptr<T, R> const&, nullptr_t) noexcept;
template <class T, class R>
bool operator != (retain_ptr<T, R> const&, nullptr_t) noexcept;
template <class T, class R>
bool operator >= (retain_ptr<T, R> const&, nullptr_t) noexcept;
template <class T, class R>
bool operator <= (retain_ptr<T, R> const&, nullptr_t) noexcept;
template <class T, class R>
bool operator > (retain_ptr<T, R> const&, nullptr_t) noexcept;
template <class T, class R>
bool operator < (retain_ptr<T, R> const&, nullptr_t) noexcept;

template <class T, class R>
bool operator == (nullptr_t, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator != (nullptr_t, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator >= (nullptr_t, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator <= (nullptr_t, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator > (nullptr_t, retain_ptr<T, R> const&) noexcept;
template <class T, class R>
bool operator < (nullptr_t, retain_ptr<T, R> const&) noexcept;

```

```

template <class T, class R> void swap (retain_ptr<T, R>& x, retain_ptr<T, R>& y) noexcept

```

effects: Calls `x.swap(y)`

```

template <class T, class R> bool operator == (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

```

returns: `x.get() == y.get()`

```

template <class T, class R> bool operator != (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

```

returns: `x.get() != y.get()`

```

template <class T, class R> bool operator >= (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

```

returns: `not (x < y)`

```

template <class T, class R> bool operator <= (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

```

returns: `not (y < x)`

```

template <class T, class R> bool operator > (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

```

returns: `y < x`

```

template <class T, class R> bool operator < (retain_ptr<T, R> const& x, retain_ptr<T, R> const& y) noexcept;

```

returns: `x.get() < y.get()`

```

template <class T, class R> bool operator == (retain_ptr<T, R> const& x, nullptr_t) noexcept

```

```

template <class T, class R> bool operator == (nullptr_t, retain_ptr<T, R> const& x) noexcept

```

returns: `not x`

```
template <class T, class R> bool operator != (retain_ptr<T, R> const& x, nullptr_t) noexcept template <class T,
class R> bool operator != (nullptr_t, retain_ptr<T, R> const& x) noexcept
```

returns: `bool(x)`

```
template <class T, class R> bool operator >= (retain_ptr<T, R> const& x, nullptr_t) noexcept
```

```
template <class T, class R> bool operator >= (nullptr_t, retain_ptr<T, R> const& x) noexcept
```

returns: The first function template returns **not (x < nullptr)**. The second function template returns **not (nullptr < x)**

```
template <class T, class R> bool operator <= (retain_ptr<T, R> const& x, nullptr_t) noexcept
```

```
template <class T, class R> bool operator <= (nullptr_t, retain_ptr<T, R> const& x) noexcept
```

returns: The first function template returns **not (nullptr < x)**. The second function template returns **not (x < nullptr)**.

```
template <class T, class R> bool operator > (retain_ptr<T, R> const& x, nullptr_t) noexcept
```

```
template <class T, class R> bool operator > (nullptr_t, retain_ptr<T, R> const& x) noexcept
```

returns: The first function template returns **nullptr < x**. The second function template returns **x < nullptr**.

```
template <class T, class R> bool operator < (retain_ptr<T, R> const& x, nullptr_t) noexcept
```

```
template <class T, class R> bool operator < (nullptr_t, retain_ptr<T, R> const& x) noexcept
```

returns: The first function template returns **x.get() < nullptr**. The second function template returns **nullptr < x.get()**.

6 Examples

Some C APIs that would benefit from **retain_ptr<T>** are:

- OpenCL
- Mono
- Python
- ObjC Runtime
- Grand Central Dispatch

Inside the github (https://github.com/slurps-mad-rips/retain_ptr) repository is an example of using **retain_ptr** with Python. Below is a basic example of how a future/promise would be implemented without a void specialization (as this is not done for completeness).

6.1 Futures

```
template <class T>
struct shared_state : atomic_reference_counter {
    bool empty () const noexcept { return get_if<0>(this->obj); }
    T get () const noexcept(false) {
        if (auto ptr = get_if<1>(this->obj)) { return move(*ptr); }
        rethrow_exception(get<2>(this->obj));
    }
    variant<monostate, T, exception_ptr> obj;
};

template <class T>
struct promise {
    using value_type = T;

    future<T> get_future () const noexcept { return this->state; }

    template <class U>
    void set_value (U&& value) {
        this->state.template emplace<1>(forward<U>(value));
    }

    void set_exception (exception_ptr ptr) noexcept {
        this->state.template emplace<2>(ptr);
    }

private:
    retain_ptr<shared_state<T>> state { new shared_state<T>() };
};

template <class T>
struct future {
    friend promise<T>;

    bool valid () const noexcept { return this->state->empty(); }
    T get () noexcept(false) {
        if (not this->valid()) { throw future_error(future_errc::no_state); }
        return this->state->get();
    }

private:
    future (retain_ptr<shared_state<T>> const& state) : state(state) { }
    retain_ptr<shared_state<T>> state;
};
```


7 Acknowledgements

A special thanks to Jackie Kay and Brittany Friedman for offering advice and pushing me to finally sit down and write this proposal.

8 References

- [1] http://www.boost.org/doc/libs/1_62_0/libs/smart_ptr/intrusive_ptr.html
(http://www.boost.org/doc/libs/1_62_0/libs/smart_ptr/intrusive_ptr.html)
- [2] <https://msdn.microsoft.com/en-us/library/br244983.aspx> (<https://msdn.microsoft.com/en-us/library/br244983.aspx>)
- [3] <https://github.com/WebKit/webkit/blob/master/Source/WTF/wtf/RefPtr.h>
(<https://github.com/WebKit/webkit/blob/master/Source/WTF/wtf/RefPtr.h>)